Lab 05

Goal

Add a “removed” flag to your tasks using if/else, then diagnose a classic .gitignore mistake that trips up almost every developer at some point.

Add a way to remove a task

- Open Main.java. Right under your task1 and task2 variables, add a boolean variable for each task that tracks whether it's been removed:

```
String task1 = "Buy milk";
String task2 = "Walk the dog";
boolean task1Removed = false;
boolean task2Removed = true;
```

- Update your println statements so a task only prints if it hasn't been removed:

```
System.out.println("My Tasks:");
if (!task1Removed) {
    System.out.println("1. " + task1);
}
if (!task2Removed) {
    System.out.println("2. " + task2);
}
```

- **Predict first:** based on the values above, which task(s) do you expect to print? Write your guess, then compile and run to check:

I expect task 1 to print and task 2 not to print.

```
javac Main.java
java Main
```

- Compile, add, commit, and push the changes

```
git add .
git commit -m "Add removed flag for tasks"
git push
```

A build artifact sneaks in (and sets up a common mistake)

- Check status — notice Main.class shows up as untracked every time you compile

```
git status
```

- On purpose, commit it anyway, as if you didn't notice (this sets up the mistake we're about to diagnose)

```
git add .
git commit -m "oops - accidentally committed class file"
git push
```

Diagnose: .gitignore does NOT un-track files that are already committed

- Now create a .gitignore file with the following contents:

```
*.class
```

- **Predict first:** now that .class files are listed in .gitignore, will Main.class disappear from git status? Write down your guess before running the next command.

It will not

```
git status
```

- You should find that Main.class is STILL tracked, even though it's in .gitignore — that's the trap. In your own words, explain in your submission why this happens.

This happens because .gitignore does not retroactively go back and un-track files.

- Fix it by explicitly telling Git to stop tracking the already-committed file:

```
git rm --cached Main.class
git add .
git commit -m "Stop tracking class file"
git status
```

- Confirm git status is now clean and that future .class files won't be tracked, even after recompiling

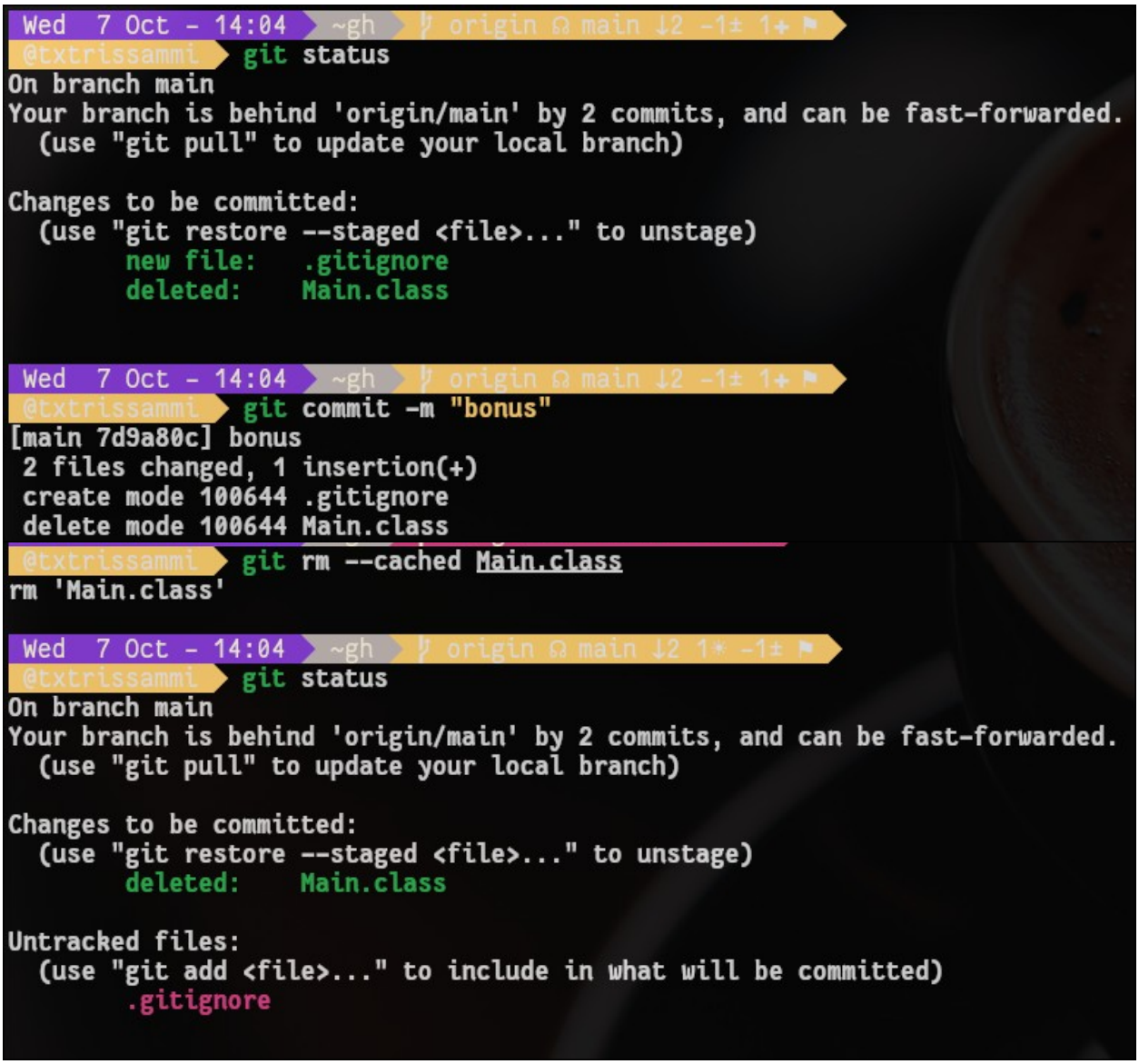
Bonus challenge (if time permits)

- Before you push the fix above, try undoing it with git reset --hard HEAD^, look at git status again to see Main.class reappear as tracked, then redo the fix. This shows how reset --hard can undo more than you intend — always double check what commit you're resetting past.

Submitting your solution

- Upload a PDF with screenshots of each step above, including your predictions and explanation for the .gitignore trap
- Use the following Naming convention: Lab05_[Last Name].pdf

Example: Lab05_Maxwell.pdf


Wed 7 Oct - 14:04 ~gh origin main ↓2 -1± 1+
@txtrissammi git status
On branch main
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
new file: .gitignore
deleted: Main.class
Wed 7 Oct - 14:04 ~gh origin main ↓2 -1± 1+
@txtrissammi git commit -m "bonus"
[main 7d9a80c] bonus
2 files changed, 1 insertion(+)
create mode 100644 .gitignore
delete mode 100644 Main.class
@txtrissammi git rm --cached Main.class
rm 'Main.class'
Wed 7 Oct - 14:04 ~gh origin main ↓2 1* -1± 1+
@txtrissammi git status
On branch main
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
deleted: Main.class
Untracked files:
(use "git add <file>..." to include in what will be committed)
.gitignore